# Markdown Studio — Feature Demo

Open this file and run **Markdown Studio: Preview** (`Cmd+Shift+P`).

---

## 1. Markdown Rendering

**Bold**, *italic*, ~~strikethrough~~, `inline code`

> Blockquotes work too.

| Feature | Status |
|---|---|
| Markdown | ✅ |
| Mermaid | ✅ |
| PlantUML | ✅ |
| SVG | ✅ |
| PDF Export | ✅ |

1. Ordered list item
2. Another item
3. Third item

- Unordered item
- Another item

- Item

- 日本語
  - 2byte文字テスト

---

## 2. Mermaid Diagrams

### Markdown Studio Architecture

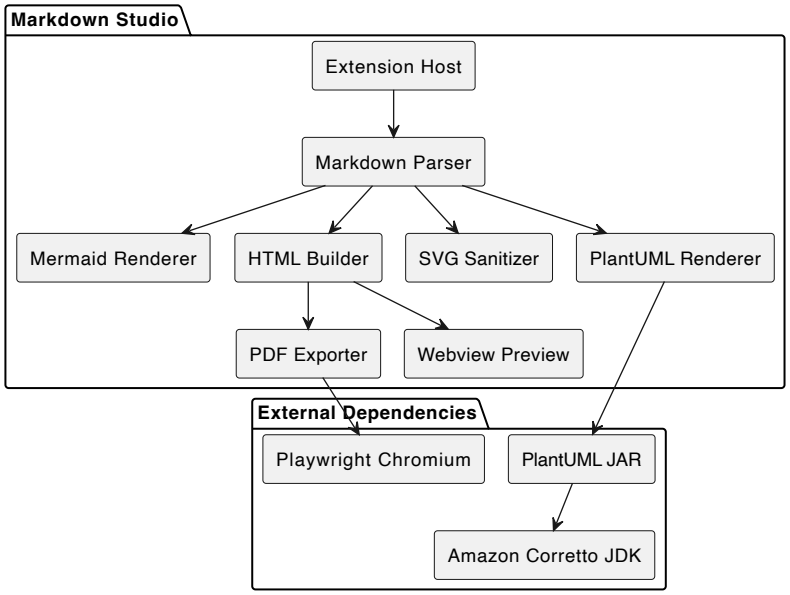
### Extension Activation Flow

---

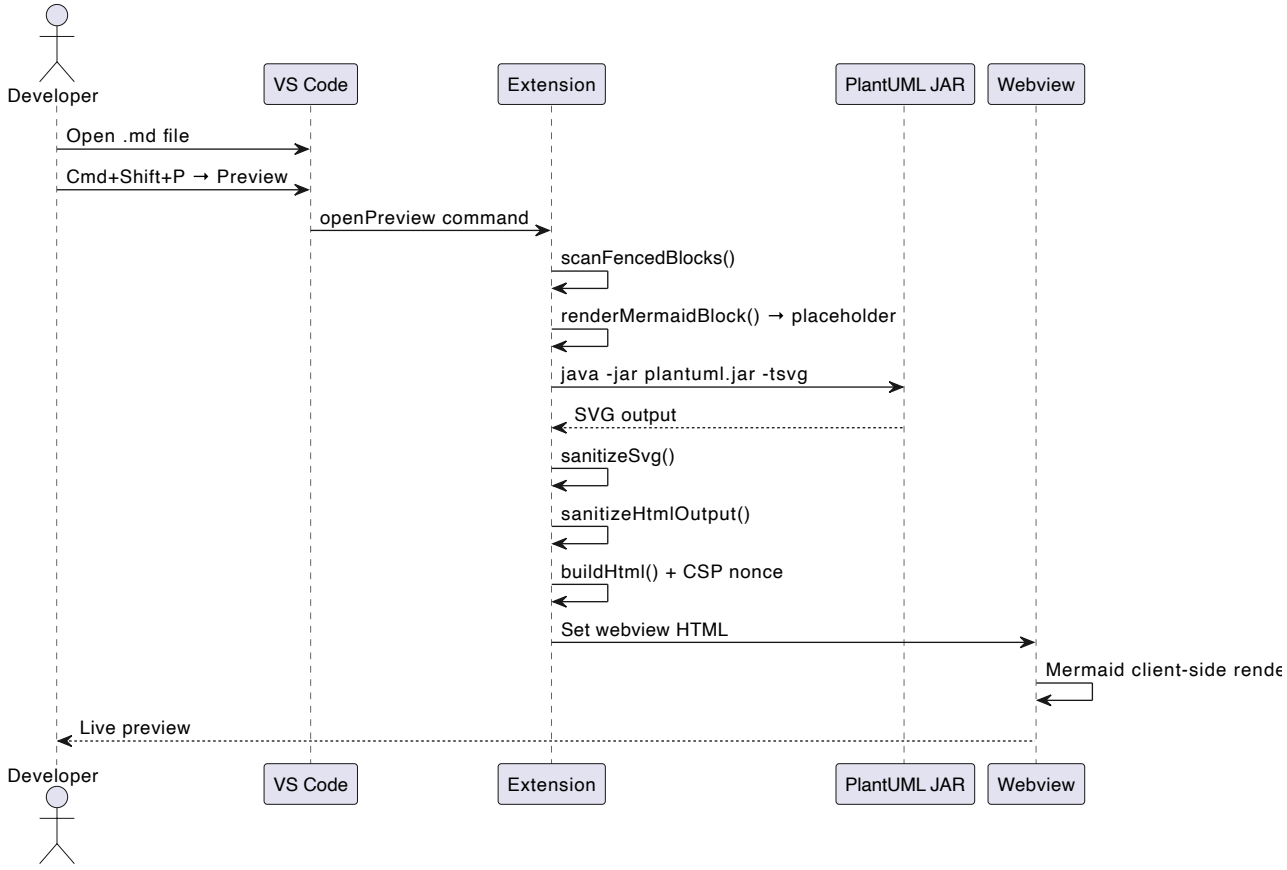
## 3. PlantUML Diagrams

Rendered locally via bundled JAR. No remote server.

### Extension Component Diagram



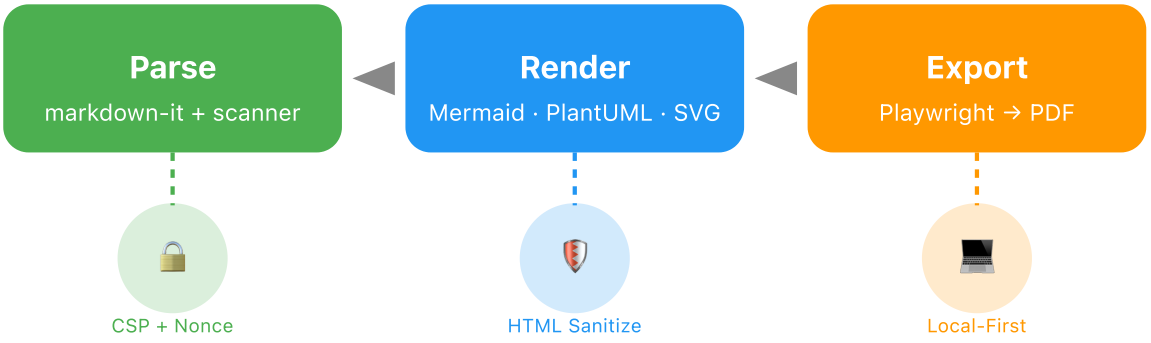

### Document Processing Sequence




---

## 4. Inline SVG (Sanitized)

Dangerous elements (`<script>`, `<foreignObject>`, event handlers) are stripped automatically.




---

## 5. Syntax Highlighting

```python
import * as vscode from 'vscode';

export function activate(context: vscode.ExtensionContext): void {
  const depManager = new DependencyManager();
  const status = await depManager.ensureAll(context);

  context.subscriptions.push(
    vscode.commands.registerCommand('markdownStudio.openPreview', async () => {
      await openPreviewCommand(context);
    }),
    vscode.commands.registerCommand('markdownStudio.exportPdf', async () => {
      await exportPdfCommand(context);
    })
  );
}
```

```yaml
{
  "markdownStudio.plantuml.mode": "bundled-jar",
  "markdownStudio.java.path": "java",
  "markdownStudio.export.pageFormat": "A4",
  "markdownStudio.security.blockExternalLinks": true
}
```

```python
# PlantUML rendering is also useful for Python projects
from dataclasses import dataclass

@dataclass
class DiagramConfig:
    mode: str = "bundled-jar"
    java_path: str = "java"
    timeout_ms: int = 15000
```

---

## 6. Images and Security

Markdown Studio is local-first. The default security policy (`blockExternalLinks = true`) blocks external resources while allowing local files.

### ✅ Local image (relative path — works)

- local svg Local SVG
- icon.png

Extension Icon.png

- icon.svg

Extension Icon.svg

Relative paths from the Markdown file are resolved and displayed correctly.

### ✅ Local image (absolute path — works)

Images with absolute file paths also work in the preview.

### ❌ External image (blocked by policy)

External image blocked by policy.

External URLs (`https://...`) are blocked by default. The image above shows a policy notice instead.

### ❌ External link (blocked by policy)

External link — blocked by default ✋

External links are replaced with a blocked notice in the preview.

### How to allow external resources

Set `markdownStudio.security.blockExternalLinks` to `false` in VS Code settings to allow external images and links. This is not recommended for corporate/secure environments.

### Security Summary

| Resource Type | Default | Configurable |
|---|---|---|
| Local images (relative path) | ✅ Allowed | — |
| Local images (absolute path) | ✅ Allowed | — |
| External images (https://) | ❌ Blocked | `blockExternalLinks: false` |
| External links (https://) | ❌ Blocked | `blockExternalLinks: false` |
| Inline SVG | ✅ Allowed | — |
| Mermaid diagrams | ✅ Allowed | — |
| PlantUML diagrams | ✅ Allowed | — |

---

## 7. Theme Adaptability

Switch between **light** and **dark** mode in VS Code (`Cmd+K Cmd+T`) to see how the preview adapts:

- Mermaid diagrams automatically switch between light and dark themes
- SVG elements use colors chosen for visibility in both themes
- Code blocks use theme-aware syntax highlighting
- PlantUML output receives CSS overrides for dark mode

---

## 8. Diagram Type Catalog

All diagram types below are verified to render correctly with the bundled PlantUML + Smetana engine (no Graphviz required).

▶ Class Diagram
▶ Activity Diagram
▶ State Diagram
▶ Use Case Diagram
▶ Timing Diagram

- ▶ Mind Map
- ▶ Gantt Chart
- ▶ Object Diagram
- ▶ Mermaid: Pie Chart
- ▶ Mermaid: Git Graph
- ▶ SVG: Basic Shapes
- ▶ SVG: Lines and Paths
- ▶ SVG: Text and Styling
- ▶ SVG: Dashboard Layout

---

## 9. PDF Export

This entire document can be exported to PDF:

1. Open this file in VS Code
2. `Cmd+Shift+P` → **Markdown Studio: Export PDF**
3. A `demo.pdf` will be generated next to this file

The PDF uses the same HTML pipeline as the preview.

---

*Markdown Studio v0.2.0*